

Exercise: Reinforcement Learning

Nguyễn Đăng Nhã

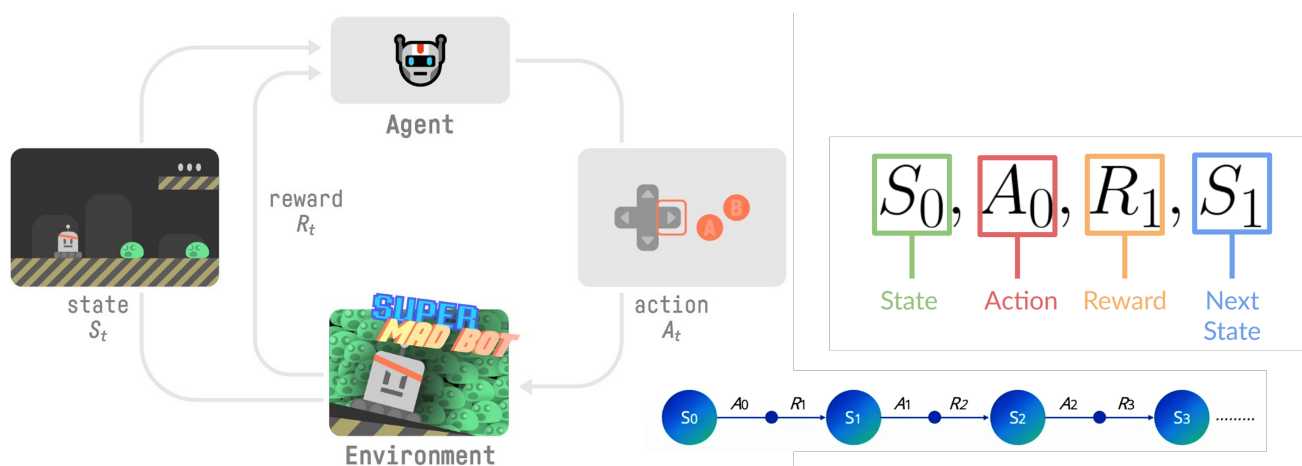
Hồ Quang Hiện

Đinh Quang Vinh

I. Giới thiệu

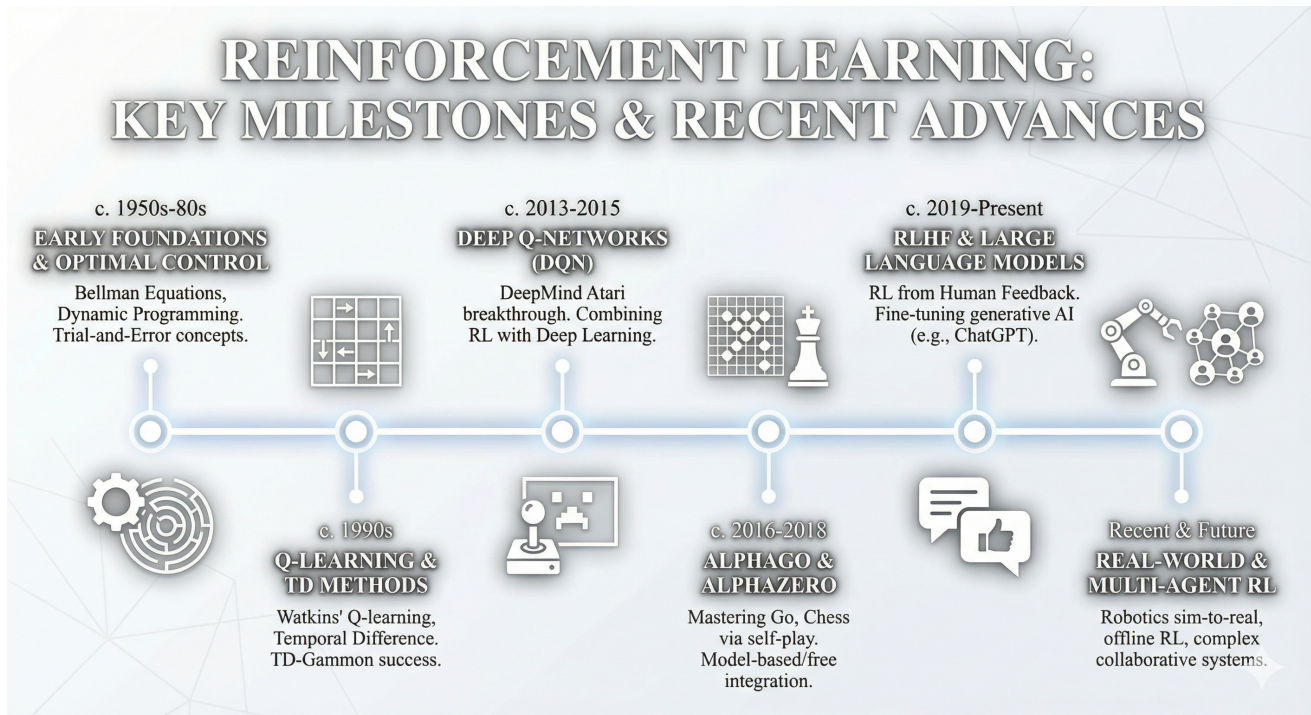
Học tăng cường là hướng tiếp cận trong trí tuệ nhân tạo nơi một tác nhân học cách ra quyết định thông qua quá trình tương tác với môi trường. Thay vì chỉ học từ các cặp dữ liệu đầu vào–đầu ra cố định, tác nhân quan sát trạng thái hiện tại, lựa chọn hành động, nhận phần thưởng, và dần điều chỉnh chiến lược để đạt được mục tiêu dài hạn. Tài liệu này trình bày nền tảng của học tăng cường theo khuôn khổ Markov Decision Process (MDP), bao gồm các thành phần cốt lõi như agent, environment, state, action, reward, transition và policy, từ đó xây dựng cái nhìn thống nhất về cách một bài toán ra quyết định tuần tự được mô hình hoá và giải quyết.

Nội dung được tổ chức theo lộ trình từ cơ bản đến thực hành. Trước hết, tài liệu làm rõ khái niệm return và hai hàm giá trị quan trọng là state-value function và action-value function, giúp người học hiểu cách đánh giá chất lượng của trạng thái và hành động trong ngữ cảnh tối ưu phần thưởng tích lũy. Trên nền tảng đó, tài liệu tiếp tục giới thiệu advantage cùng các phương pháp ước lượng giá trị tiêu biểu như Monte Carlo, TD(0), TD(λ) và Generalized Advantage Estimation (GAE), nhằm cho thấy sự khác nhau giữa các cách học từ trải nghiệm và vai trò của chúng trong việc cân bằng giữa độ chệch và phương sai.



Hình 1: Mô phỏng tổng quát quá trình tác nhân tương tác với môi trường trong học tăng cường.

Sau phần nền tảng, tài liệu chuyển sang các thuật toán học tăng cường quan trọng thuộc cả hai hướng value-based và policy-based. Người học sẽ lần lượt tiếp cận SARSA, Q-learning, Deep Q-Learning, REINFORCE, PPO và AWR để thấy cách mỗi thuật toán xây dựng quy tắc cập nhật, tận dụng dữ liệu thu thập được, và cải thiện chính sách theo những giả định khác nhau. Mục tiêu chính của tài liệu là giúp người học nắm được mạch tư duy xuyên suốt từ mô hình hoá bài toán, ước lượng giá trị, thiết kế tín hiệu học, cho đến triển khai các thuật toán hiện đại, từ đó có thể tự đọc hiểu, cài đặt, thử nghiệm và mở rộng sang các bài toán điều khiển và ra quyết định phức tạp hơn.



Hình 2: Các cột mốc quan trọng và những tiến bộ gần đây của học tăng cường.

Mục lục

I.	Giới thiệu	1
II.	Markov Decision Process (MDP)	4
III.	Return và các hàm giá trị	5
IV.	Advantage và các phương pháp ước lượng giá trị	8
IV.1.	Advantage Là Gì?	8
IV.2.	Monte Carlo (MC)	9
IV.3.	Temporal Difference: TD(0)	10
IV.4.	TD(λ)	11
IV.5.	Generalized Advantage Estimation (GAE)	11
IV.6.	So Sánh Nhanh	12
IV.7.	Flow Chung	13
V.	Một số thuật toán học tăng cường cơ bản và hiện đại	13
V.1.	SARSA	13
V.2.	Q-learning	14
V.3.	Deep Q-Learning (DQL)	15
V.4.	REINFORCE	16
V.5.	PPO	17
V.6.	AWR	18
V.7.	So sánh nhanh	19
VI.	Thực hành bài toán Q-learning với Taxi-v3	20
VII.	Thực hành approximate Q-learning với MsPacman	25
VIII.	Thực hành REINFORCE với LunarLander-v3	30
IX.	Thực hành PPO với Acrobot-v1	36
X.	Câu hỏi trắc nghiệm	41
	Phụ lục	45

II. Markov Decision Process (MDP)

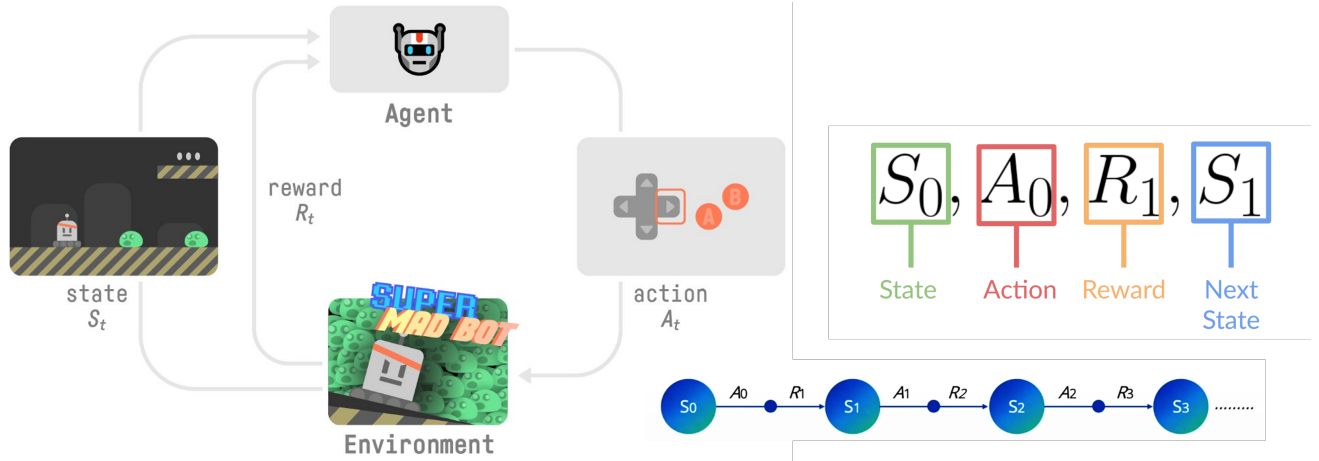
Markov Decision Process (MDP) là mô hình dùng để mô tả bài toán ra quyết định theo từng bước trong học tăng cường. Trong mô hình này, một tác nhân sẽ tương tác với môi trường, nhận phản hồi, rồi dần học cách chọn hành động tốt hơn để tối đa hoá kết quả dài hạn.

Một MDP thường gồm các thành phần chính sau:

- **Agent**: Là tác nhân ra quyết định. Đây là thành phần sẽ quan sát trạng thái hiện tại, chọn hành động, và học từ phản hồi nhận được.
- **Environment**: Là môi trường mà agent tương tác. Environment nhận hành động từ agent, sau đó trả về trạng thái mới và phần thưởng tương ứng.
- **State (s)**: Là trạng thái hiện tại của môi trường. State mô tả tình huống mà agent đang đối mặt tại một thời điểm.
- **Action (a)**: Là hành động mà agent chọn tại trạng thái hiện tại. Mỗi state có thể có một hoặc nhiều action khả thi.
- **Reward (r)**: Là tín hiệu phản hồi từ môi trường sau khi agent thực hiện action. Reward cho biết hành động đó tốt hay xấu ở mức nào.
- **Next State (s')**: Là trạng thái mới mà môi trường chuyển tới sau khi action được thực hiện.
- **Policy (π)**: Là chiến lược chọn hành động của agent. Policy cho biết tại mỗi state thì agent nên chọn action nào.
- **Transition**: Là quy tắc chuyển từ state hiện tại sang next state sau khi thực hiện action. Trong nhiều bài toán, quá trình này có thể mang tính xác suất.

Flow hoạt động của một MDP diễn ra theo vòng lặp sau:

1. **Environment** cung cấp **state hiện tại** cho **agent**.
2. **Agent** quan sát state và chọn một **action**.
3. **Environment** nhận action đó và phản hồi bằng một **reward**.
4. **Environment** đồng thời chuyển sang một **next state**.
5. **Agent** tiếp tục quan sát trạng thái mới và lặp lại quá trình trên.



Hình 3: Vòng lặp tương tác cơ bản giữa agent và environment trong MDP.

Có Thể Tóm Gọn Flow Của MDP Như Sau:

$$s \rightarrow a \rightarrow r, s'$$

Tức là agent quan sát trạng thái s , chọn hành động a , rồi môi trường trả về phần thưởng r và trạng thái mới s' .

Khi hiểu rõ các thành phần này, người học sẽ dễ tiếp cận hơn các khái niệm tiếp theo như return, value function, advantage, cũng như các thuật toán như SARSA, Q-learning, DQL, REINFORCE và PPO.

III. Return và các hàm giá trị

Sau khi mô hình hoá bài toán bằng MDP, câu hỏi tiếp theo là: làm sao biết một trạng thái hay một hành động là tốt? Để trả lời điều đó, học tăng cường sử dụng các khái niệm **return** và **value function**. Đây là nền tảng để agent không chỉ nhìn vào phần thưởng ngay trước mắt, mà còn đánh giá được lợi ích dài hạn trong tương lai.

Return là tổng phần thưởng tích lũy mà agent nhận được từ một thời điểm trở đi. Thay vì chỉ quan tâm reward ở bước hiện tại, return cộng dồn nhiều reward trong tương lai để phản ánh mục tiêu dài hạn. Nếu dùng hệ số chiết khấu γ , return tại thời điểm t được viết là:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

Trong đó:

- G_t : return tại thời điểm t .
- $r_{t+1}, r_{t+2}, \dots$: các phần thưởng nhận được ở những bước sau.
- $\gamma \in [0, 1]$: hệ số chiết khấu, dùng để giảm ảnh hưởng của reward ở quá xa trong tương lai.

Nếu γ lớn, agent sẽ quan tâm nhiều hơn đến lợi ích dài hạn. Nếu γ nhỏ, agent sẽ thiên về phần thưởng ngắn hạn.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

Return: cumulative reward Gamma: discount rate

Trajectory (read Tau)
Sequence of states and actions

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Hình 4: Minh họa công thức return tích lũy và vai trò của hệ số chiết khấu γ trong học tăng cường.

Ví dụ: giả sử tại thời điểm t , agent nhận được chuỗi reward:

$$r_{t+1} = 2, \quad r_{t+2} = 3, \quad r_{t+3} = 1$$

và chọn:

$$\gamma = 0.9$$

Khi đó:

$$G_t = 2 + 0.9 \cdot 3 + 0.9^2 \cdot 1$$

$$G_t = 2 + 2.7 + 0.81 = 5.51$$

Điều này nghĩa là nếu nhìn theo mục tiêu dài hạn, giá trị phần thưởng tích lũy từ thời điểm t là 5.51, chứ không chỉ là reward trước mắt bằng 2.

Từ return, ta xây dựng hai hàm giá trị quan trọng để đánh giá trạng thái và hành động.

- **State-value function, ký hiệu $V^\pi(s)$:** cho biết nếu agent đang ở trạng thái s và tiếp tục hành động theo policy π , thì return kỳ vọng mà agent sẽ nhận được là bao nhiêu. Nói đơn giản, $V^\pi(s)$ đo độ tốt của một trạng thái.
- **Action-value function, ký hiệu $Q^\pi(s, a)$:** cho biết nếu agent đang ở trạng thái s , thực hiện hành động a , rồi sau đó tiếp tục theo policy π , thì return kỳ vọng là bao nhiêu. Nói đơn giản, $Q^\pi(s, a)$ đo độ tốt của một hành động tại một trạng thái cụ thể.

Hai công thức định nghĩa quan trọng là:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

Ý nghĩa của hai công thức này là:

- $V^\pi(s)$: nếu chỉ biết agent đang ở state s , thì tương lai kỳ vọng tốt đến mức nào.
- $Q^\pi(s, a)$: nếu biết agent đang ở state s và chọn action a , thì tương lai kỳ vọng tốt đến mức nào.

Có thể hiểu trực quan như sau:

- $V(s)$ **trả lời câu hỏi**: “Đứng ở trạng thái này thì nhìn chung tương lai tốt đến mức nào?”
- $Q(s, a)$ **trả lời câu hỏi**: “Nếu đang ở trạng thái này và chọn hành động này, thì tương lai tốt đến mức nào?”

Ví dụ: giả sử trong một state s , agent có hai hành động:

- Action a_1 : đi sang trái, với giá trị:

$$Q(s, a_1) = 4$$

- Action a_2 : đi sang phải, với giá trị:

$$Q(s, a_2) = 7$$

Khi đó ta hiểu rằng hành động a_2 tốt hơn hành động a_1 tại state s , vì nếu chọn a_2 thì return kỳ vọng lớn hơn.

Nếu policy hiện tại chọn các hành động trong state s theo một cách nào đó, chẳng hạn giá trị trung bình của state đó là:

$$V^\pi(s) = 5.5$$

thì có thể hiểu rằng đứng tại state s , tương lai kỳ vọng trung bình là 5.5. Tuy nhiên, nếu chọn đúng hành động tốt hơn thì agent có thể đạt giá trị cao hơn mức trung bình đó.

Một mối liên hệ trực quan giữa $V(s)$ và $Q(s, a)$ là:

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$$

Tức là giá trị của state là trung bình có trọng số của các action-value theo policy hiện tại.

Ví dụ: giả sử tại state s , policy chọn:

$$\pi(a_1|s) = 0.25, \quad \pi(a_2|s) = 0.75$$

và:

$$Q(s, a_1) = 4, \quad Q(s, a_2) = 8$$

Khi đó:

$$V^\pi(s) = 0.25 \cdot 4 + 0.75 \cdot 8$$

$$V^\pi(s) = 1 + 6 = 7$$

Điều này nghĩa là nếu agent đứng ở state s và hành động theo policy hiện tại, thì return kỳ vọng trung bình là 7.

Flow suy nghĩ của agent khi dùng các hàm giá trị có thể hình dung như sau:

1. Agent đang ở một **state** hiện tại.
2. Agent muốn biết trạng thái này tốt đến đâu hoặc từng hành động tại đây tốt đến đâu.
3. Return được dùng làm mục tiêu dài hạn để đánh giá.
4. Từ return, agent học ra các hàm $V(s)$ và $Q(s, a)$.
5. Dựa trên các giá trị này, agent dần chọn hành động tốt hơn.

Điểm quan trọng là reward chỉ là tín hiệu tức thời, còn return và value function mới giúp agent nhìn xa hơn. Nhờ đó, agent có thể chấp nhận một hành động chưa có lợi ngay lập tức nếu biết rằng nó sẽ dẫn tới phần thưởng tốt hơn trong tương lai.

Tóm lại:

- **Return** cho biết tổng phần thưởng tích lũy trong tương lai.
- $V(s)$ cho biết một trạng thái tốt đến mức nào.
- $Q(s, a)$ cho biết một hành động tại trạng thái đó tốt đến mức nào.

Phần này là nền tảng để đi tiếp sang các khái niệm như **advantage**, cũng như các phương pháp ước lượng giá trị như **Monte Carlo**, **TD(0)**, **TD(λ)** và **GAE**.

IV. Advantage và các phương pháp ước lượng giá trị

Sau khi đã có **return**, **state-value function** và **action-value function**, câu hỏi tiếp theo là: làm sao ước lượng các đại lượng này từ dữ liệu thực tế? Trong học tăng cường, agent không biết sẵn giá trị đúng của trạng thái hay hành động, nên phải học dần từ các trajectory đã thu thập được. Phần này giới thiệu khái niệm **advantage** và các phương pháp ước lượng phổ biến gồm **Monte Carlo (MC)**, **TD(0)**, **TD(λ)** và **GAE**.

IV.1. Advantage Là Gì?

Advantage đo xem một hành động có tốt hơn mức trung bình tại trạng thái hiện tại hay không. Công thức cơ bản là:

$$A(s, a) = Q(s, a) - V(s)$$

Trong đó:

- $Q(s, a)$: Giá trị của việc chọn hành động a tại trạng thái s .
- $V(s)$: Giá trị trung bình khi đứng ở trạng thái s .

- $A(s, a)$: Mức chênh lệch giữa hành động cụ thể và mức trung bình đó.

Ý nghĩa của advantage:

- Nếu $A(s, a) > 0$, hành động đó **tốt hơn trung bình**.
- Nếu $A(s, a) < 0$, hành động đó **kém hơn trung bình**.
- Nếu $A(s, a) = 0$, hành động đó gần bằng mức kỳ vọng thông thường.

Ví dụ: giả sử tại trạng thái s , ta có:

$$Q(s, a) = 8, \quad V(s) = 5$$

Khi đó:

$$A(s, a) = 8 - 5 = 3$$

Điều này có nghĩa là hành động a tốt hơn mức trung bình tại trạng thái đó là 3 đơn vị giá trị. Ngược lại, nếu:

$$Q(s, a) = 4, \quad V(s) = 5$$

thì:

$$A(s, a) = 4 - 5 = -1$$

Khi đó hành động này kém hơn mức trung bình.

IV.2. Monte Carlo (MC)

Monte Carlo ước lượng giá trị bằng cách đợi đến cuối episode, rồi cộng toàn bộ reward để tính return thật.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

Ở đây:

- G_t : Return tại thời điểm t .
- $r_{t+1}, r_{t+2}, \dots$: Các reward trong tương lai.
- γ : Hệ số chiết khấu, giúp giảm ảnh hưởng của reward quá xa.

Ví dụ: giả sử từ thời điểm t agent nhận được chuỗi reward:

$$r_{t+1} = 2, \quad r_{t+2} = 3, \quad r_{t+3} = 1$$

và:

$$\gamma = 0.9$$

Khi đó:

$$G_t = 2 + 0.9 \cdot 3 + 0.9^2 \cdot 1$$

$$G_t = 2 + 2.7 + 0.81 = 5.51$$

Điều này nghĩa là tổng phần thưởng tích lũy từ thời điểm t là 5.51.

Đặc điểm của Monte Carlo:

- Dễ hiểu vì bám sát đúng định nghĩa return.
- Không cần ước lượng trung gian.
- Nhưng phải đợi hết episode mới cập nhật được.
- Thường có phương sai cao.

IV.3. Temporal Difference: TD(0)

TD(0) không cần đợi đến cuối episode. Thay vào đó, nó cập nhật ngay ở từng bước bằng cách dùng reward hiện tại cộng với giá trị ước lượng của trạng thái kế tiếp.

Công thức mục tiêu TD(0):

$$\text{Target} = r_{t+1} + \gamma V(s_{t+1})$$

Sai số TD:

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Trong đó:

- $V(s_t)$: Giá trị hiện tại ta đang ước lượng cho state s_t .
- $V(s_{t+1})$: Giá trị ước lượng của state tiếp theo.
- δ_t : Sai số TD, cho biết ước lượng hiện tại lệch bao nhiêu.

Ví dụ: giả sử:

$$r_{t+1} = 2, \quad \gamma = 0.9, \quad V(s_{t+1}) = 4, \quad V(s_t) = 5$$

Khi đó:

$$\text{Target} = 2 + 0.9 \cdot 4 = 5.6$$

và:

$$\delta_t = 5.6 - 5 = 0.6$$

Sai số dương cho thấy giá trị hiện tại $V(s_t) = 5$ đang bị ước lượng hơi thấp, và nên tăng lên.

Đặc điểm của TD(0):

- Cập nhật nhanh hơn MC.
- Không cần chờ hết episode.
- Học online tốt hơn.
- Nhưng vì dùng ước lượng để học từ ước lượng nên có thể bị chệch.

IV.4. TD(λ)

TD(λ) là cách kết hợp giữa Monte Carlo và TD(0). Thay vì chỉ nhìn một bước như TD(0), nó kết hợp nhiều mức nhìn trước khác nhau.

Ý tưởng chính:

- Nếu $\lambda = 0$, TD(λ) gần giống **TD(0)**.
- Nếu $\lambda \rightarrow 1$, TD(λ) gần giống **Monte Carlo**.
- Nếu $0 < \lambda < 1$, ta có một mức cân bằng giữa hai cách trên.

TD(λ) thường được hiểu là tổ hợp có trọng số của nhiều return khác nhau:

$$G_t^\lambda = (1 - \lambda) \left(G_t^{(1)} + \lambda G_t^{(2)} + \lambda^2 G_t^{(3)} + \dots \right)$$

Trong đó:

- $G_t^{(1)}$: Return nhìn trước 1 bước.
- $G_t^{(2)}$: Return nhìn trước 2 bước.
- $G_t^{(3)}$: Return nhìn trước 3 bước.
- λ : Hệ số điều khiển mức pha trộn.

Ta không cần bắt người học nhớ ngay công thức dài này. Điều quan trọng là hiểu trực giác: **TD(λ) trộn nhiều cách nhìn ngắn hạn và dài hạn để có tín hiệu học ổn định hơn.**

Ví dụ trực quan:

- TD(0) giống như chỉ nhìn **1 bước phía trước**.
- Monte Carlo giống như nhìn **đến cuối hành trình**.
- TD(λ) giống như nhìn **nhiều khoảng khác nhau rồi gộp lại**.

IV.5. Generalized Advantage Estimation (GAE)

GAE là phương pháp thường dùng để ước lượng **advantage** trong các thuật toán policy gradient hiện đại như PPO. Nó dùng nhiều sai số TD để tạo ra một advantage mượt hơn và ổn định hơn.

Trước hết, sai số TD tại bước t là:

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

GAE tính advantage bằng cách cộng có trọng số các sai số này:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

Ý nghĩa của công thức này là:

- Không chỉ dùng sai số của một bước.
- Mà gom nhiều sai số liên tiếp trong tương lai.
- Sai số càng xa thì trọng số càng nhỏ dần.

Ví dụ đơn giản: giả sử:

$$\delta_t = 2, \quad \delta_{t+1} = 1, \quad \delta_{t+2} = 0.5$$

và:

$$\gamma = 0.9, \quad \lambda = 0.8$$

thì:

$$\begin{aligned} \hat{A}_t &\approx 2 + (0.9 \cdot 0.8) \cdot 1 + (0.9 \cdot 0.8)^2 \cdot 0.5 \\ \hat{A}_t &\approx 2 + 0.72 + 0.2592 = 2.9792 \end{aligned}$$

Như vậy advantage không chỉ dựa trên một tín hiệu đơn lẻ, mà dùng thêm thông tin từ các bước sau để ổn định hơn.

Đặc điểm của GAE:

- Giảm phương sai so với cách tính advantage đơn giản.
- Ổn định hơn khi huấn luyện policy gradient.
- Rất phổ biến trong PPO.

IV.6. So Sánh Nhanh

- **Monte Carlo:** Đợi hết episode rồi tính return thật.
- **TD(0):** Cập nhật từng bước bằng ước lượng một bước.
- **TD(λ):** Kết hợp nhiều mức nhìn trước.
- **GAE:** Ước lượng advantage ổn định hơn bằng cách cộng nhiều sai số TD.

Có thể hình dung như sau:

- **MC** nhìn **rất xa**.
- **TD(0)** nhìn **rất gần**.
- **TD(λ)** nhìn **nhiều khoảng**.
- **GAE** dùng ý tưởng đó để tính **advantage** tốt hơn.

IV.7. Flow Chung

1. Agent tương tác với environment và thu được một trajectory.
2. Từ trajectory đó, ta tính reward, return hoặc sai số TD.
3. Từ các đại lượng này, ta ước lượng value hoặc advantage.
4. Tín hiệu ước lượng đó sẽ được dùng để cập nhật policy hoặc value function.

Khi hiểu rõ phần này, người học sẽ dễ nắm bắt hơn các thuật toán như **SARSA**, **Q-learning**, **Deep Q-Learning**, **REINFORCE**, **PPO** và **AWR**, vì các thuật toán đó đều dựa vào một cách nào đó để ước lượng giá trị hoặc advantage.

V. Một số thuật toán học tăng cường cơ bản và hiện đại

Sau khi đã hiểu MDP, return, value function, advantage và các cách ước lượng giá trị, ta có thể đi vào các thuật toán cụ thể. Mỗi thuật toán đưa ra một cách khác nhau để cập nhật agent sao cho policy ngày càng tốt hơn. Có thuật toán học giá trị của hành động, có thuật toán học trực tiếp policy, và cũng có thuật toán kết hợp cả hai hướng. Trong phần này, ta lần lượt xét các thuật toán tiêu biểu gồm **SARSA**, **Q-learning**, **Deep Q-Learning (DQL)**, **REINFORCE**, **PPO** và **AWR**.

V.1. SARSA

SARSA là một thuật toán **on-policy**. Điều đó nghĩa là nó học từ chính cách hành động mà agent đang dùng để tương tác với môi trường. Tên SARSA đến từ chuỗi:

$$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$$

tức là tại trạng thái hiện tại s_t , agent chọn hành động a_t , nhận reward r_{t+1} , chuyển sang trạng thái mới s_{t+1} , rồi tiếp tục chọn hành động tiếp theo a_{t+1} .

Công thức cập nhật quan trọng của SARSA là:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

Trong đó:

- α : learning rate.
- γ : discount factor.
- $Q(s_t, a_t)$: giá trị hiện tại của hành động a_t tại trạng thái s_t .

- $Q(s_{t+1}, a_{t+1})$: giá trị của hành động tiếp theo mà policy hiện tại thực sự chọn.

Ví dụ: giả sử:

$$Q(s_t, a_t) = 5, \quad r_{t+1} = 2, \quad \gamma = 0.9, \quad Q(s_{t+1}, a_{t+1}) = 4, \quad \alpha = 0.1$$

Khi đó target là:

$$2 + 0.9 \cdot 4 = 5.6$$

Và cập nhật sẽ là:

$$Q(s_t, a_t) \leftarrow 5 + 0.1(5.6 - 5) = 5.06$$

Nghĩa là giá trị của hành động này được tăng nhẹ lên vì kết quả nhận được tốt hơn ước lượng cũ.

Cách hoạt động ngắn gọn:

1. Agent đang ở state s_t và chọn action a_t .
2. Environment trả về reward r_{t+1} và state mới s_{t+1} .
3. Agent tiếp tục chọn action a_{t+1} theo policy hiện tại.
4. Dùng chính action đó để cập nhật $Q(s_t, a_t)$.

Vì SARSA học từ hành động thật sự sẽ làm tiếp theo, nên nó thường “thận trọng” hơn trong các môi trường có rủi ro.

V.2. Q-learning

Q-learning là một thuật toán **off-policy**. Nó không cập nhật theo hành động mà agent thực sự chọn tiếp theo, mà cập nhật theo hành động tốt nhất có thể ở trạng thái kế tiếp.

Công thức cập nhật quan trọng là:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Điểm khác với SARSA là ở chỗ:

$$Q(s_{t+1}, a_{t+1}) \quad \text{được thay bằng} \quad \max_{a'} Q(s_{t+1}, a')$$

Tức là Q-learning luôn giả sử rằng từ trạng thái tiếp theo, agent sẽ chọn hành động tốt nhất.

Ví dụ: giả sử:

$$Q(s_t, a_t) = 5, \quad r_{t+1} = 2, \quad \gamma = 0.9, \quad \max_{a'} Q(s_{t+1}, a') = 6, \quad \alpha = 0.1$$

Khi đó target là:

$$2 + 0.9 \cdot 6 = 7.4$$

Và cập nhật:

$$Q(s_t, a_t) \leftarrow 5 + 0.1(7.4 - 5) = 5.24$$

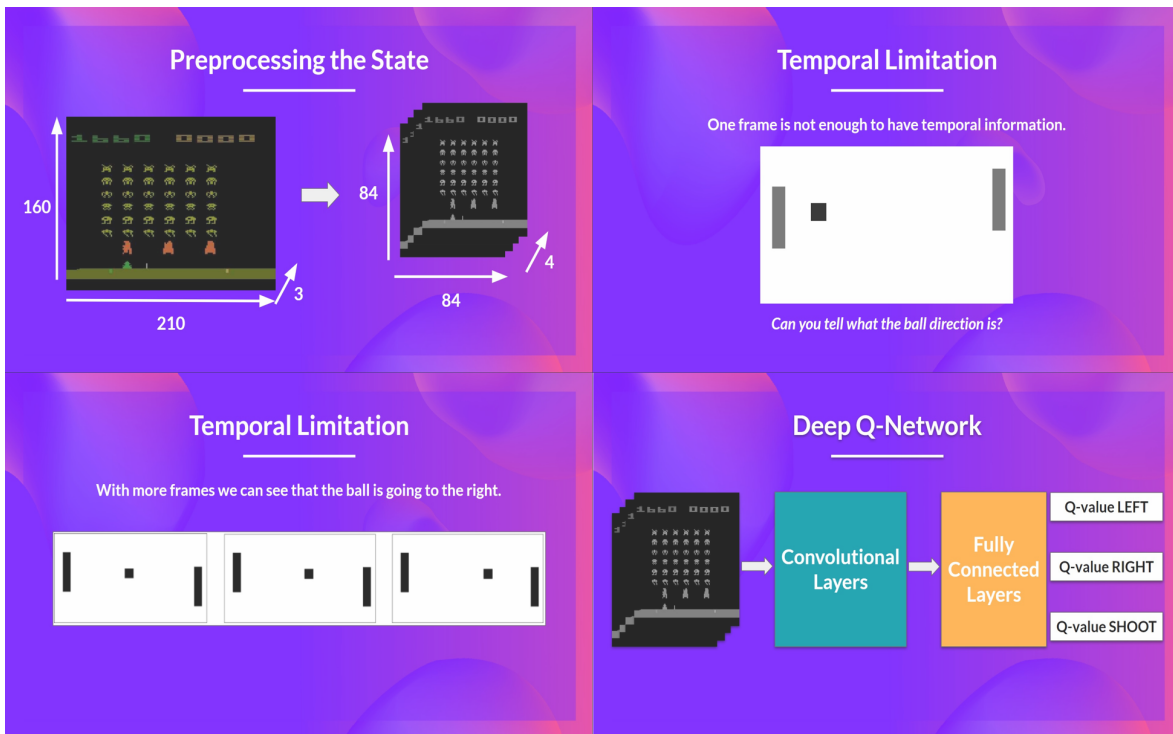
Q-value tăng mạnh hơn SARSA vì thuật toán đang nhìn theo hành động tốt nhất có thể ở bước sau.

Cách hoạt động ngắn gọn:

1. Agent thực hiện action tại state hiện tại.
2. Nhận reward và state mới.
3. Xem ở state mới thì action nào có Q-value lớn nhất.
4. Dùng giá trị lớn nhất đó để cập nhật Q-value hiện tại.

Q-learning thường “tham vọng” hơn SARSA vì nó học theo hướng tối ưu nhất ở bước tiếp theo.

V.3. Deep Q-Learning (DQL)



Hình 5: Minh hoạ ý tưởng của Deep Q-Learning: tiền xử lý trạng thái, ghép nhiều frame để nắm thông tin thời gian, và dự đoán Q-value bằng mạng tích chập.

Khi không gian trạng thái quá lớn, ta không thể lưu bảng Q cho mọi cặp (s, a) . Khi đó, ta dùng một mạng neural để xấp xỉ hàm $Q(s, a)$. Cách làm này được gọi là **Deep Q-Learning** hay **DQN**.

Mục tiêu học của DQL vẫn dựa trên ý tưởng của Q-learning:

$$y = r_{t+1} + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a')$$

Hàm mất mát thường là:

$$\mathcal{L}(\theta) = (y - Q_{\theta}(s_t, a_t))^2$$

Trong đó:

- Q_{θ} : mạng chính đang được huấn luyện.
- Q_{target} : mạng target dùng để tạo target ổn định hơn.
- y : giá trị mục tiêu.

Ví dụ: giả sử mạng hiện tại dự đoán:

$$Q_{\theta}(s_t, a_t) = 4.5$$

và target tính được là:

$$y = 6.0$$

thì loss là:

$$\mathcal{L} = (6.0 - 4.5)^2 = 2.25$$

Mạng sẽ được cập nhật để đưa dự đoán $Q_{\theta}(s_t, a_t)$ gần hơn với 6.0.

Cách hoạt động ngắn gọn:

1. Agent thu thập các transition (s, a, r, s') .
2. Đưa state vào mạng để dự đoán Q-value.
3. Tạo target bằng reward cộng với giá trị tốt nhất ở state kế tiếp.
4. Dùng loss để cập nhật trọng số của mạng.

DQL rất quan trọng vì nó mở rộng Q-learning sang các bài toán lớn như ảnh, game Atari và nhiều môi trường phức tạp.

V.4. REINFORCE

REINFORCE là một thuật toán **policy gradient**. Thay vì học Q-table hay value trước, nó học trực tiếp policy $\pi_{\theta}(a|s)$, tức là xác suất chọn action a tại state s .

Ý tưởng chính là: nếu một action dẫn tới return cao, thì tăng xác suất chọn action đó trong tương lai.

Công thức cập nhật cốt lõi là:

$$\nabla J(\theta) = \mathbb{E} \left[G_t \nabla \log \pi_{\theta}(a_t | s_t) \right]$$

Trong đó:

- $\pi_\theta(a_t|s_t)$: xác suất policy chọn action a_t tại state s_t .
- G_t : return thu được từ thời điểm t trở đi.
- Nếu G_t lớn, action đó được khuyến khích mạnh hơn.

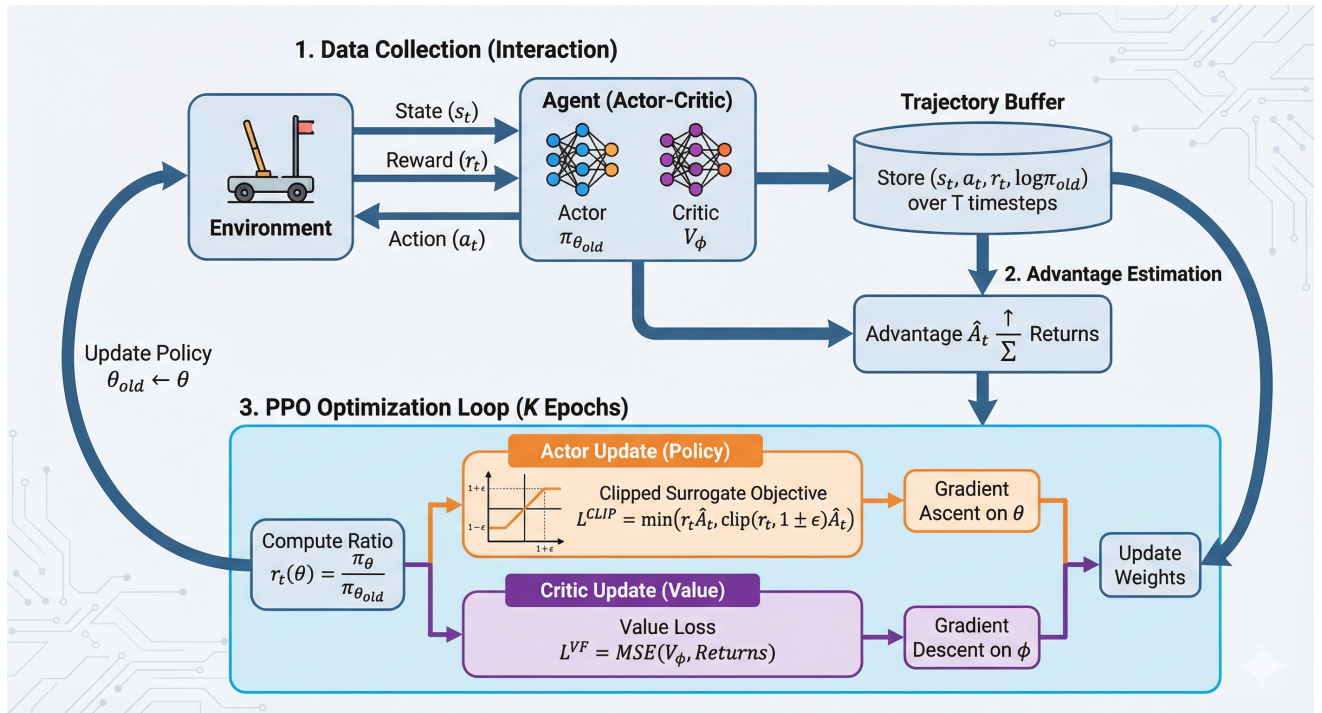
Ví dụ trực quan: giả sử agent chọn một action và sau đó nhận được return lớn. Khi đó gradient sẽ đẩy policy theo hướng tăng xác suất của action vừa chọn. Ngược lại, nếu return thấp, xác suất của action đó sẽ bị giảm.

Cách hoạt động ngắn gọn:

1. Chạy policy để sinh ra một episode.
2. Tính return cho từng bước.
3. Action nào nằm trên trajectory tốt sẽ được tăng xác suất.
4. Cập nhật trực tiếp tham số của policy.

REINFORCE đơn giản và dễ hiểu, nhưng thường có phương sai cao vì dùng return toàn quỹ đạo để cập nhật.

V.5. PPO



Hình 6: Quy trình tổng quát của PPO, từ thu thập trajectory, ước lượng advantage đến cập nhật actor-critic bằng clipped objective.

PPO là một thuật toán policy gradient hiện đại, rất phổ biến vì vừa mạnh vừa ổn định. Ý tưởng chính của PPO là cập nhật policy, nhưng không để policy mới thay đổi quá xa policy cũ chỉ trong một bước học.

Tỉ số giữa policy mới và policy cũ được viết là:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

Hàm mục tiêu nổi tiếng của PPO là:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

Trong đó:

- $r_t(\theta)$: mức thay đổi xác suất của action giữa policy mới và policy cũ.
- $\hat{A}_t$: advantage ước lượng, thường lấy từ GAE.
- ϵ : ngưỡng chặn để tránh update quá mạnh.

Ví dụ trực quan: giả sử policy mới làm xác suất của action tăng quá nhiều so với policy cũ. Khi đó hàm clip sẽ chặn bớt mức tăng này, giúp quá trình học ổn định hơn. Nếu không có clip, policy có thể thay đổi quá mạnh và làm huấn luyện mất ổn định.

Cách hoạt động ngắn gọn:

1. Dùng policy hiện tại để thu thập dữ liệu.
2. Tính value và advantage.
3. So sánh policy mới với policy cũ bằng tỉ số xác suất.
4. Chỉ cho phép cập nhật trong một khoảng an toàn.

PPO được dùng rất rộng rãi vì dễ triển khai, hiệu quả tốt và ổn định hơn REINFORCE thuần túy.

V.6. AWR

AWR là viết tắt của **Advantage-Weighted Regression**. Đây là một cách học policy bằng cách xem cập nhật policy như một bài toán hồi quy có trọng số.

Ý tưởng chính là: những action có advantage cao sẽ được gán trọng số lớn hơn, nên policy sẽ học bắt chước các action tốt mạnh hơn.

Một dạng mục tiêu thường gặp của AWR là:

$$\mathcal{L}_{\text{AWR}}(\theta) = -\mathbb{E} \left[\log \pi_\theta(a_t|s_t) \exp \left(\frac{\hat{A}_t}{\beta} \right) \right]$$

Trong đó:

- $\hat{A}_t$: advantage ước lượng.
- β : hệ số điều chỉnh độ mạnh của trọng số.
- Action có advantage càng cao thì trọng số càng lớn.

Ví dụ trực quan: giả sử có hai action trong dữ liệu:

- Action A có advantage là 3.
- Action B có advantage là 0.5.

Khi đó policy sẽ học mạnh hơn từ action A vì nó tốt hơn rõ ràng so với mức trung bình.

Cách hoạt động ngắn gọn:

1. Thu thập dữ liệu hoặc dùng dữ liệu sẵn có.
2. Ước lượng value và advantage.
3. Gán trọng số lớn cho các action có advantage cao.
4. Huấn luyện policy theo kiểu hồi quy có trọng số.

AWR dễ hiểu ở chỗ nó biến việc cải thiện policy thành bài toán “học bắt chước những hành động tốt”, nhưng không bắt chước đều nhau mà ưu tiên các hành động tốt hơn.

V.7. So sánh nhanh

- **SARSA**: on-policy, cập nhật theo action thật sự chọn tiếp theo.
- **Q-learning**: off-policy, cập nhật theo action tốt nhất ở bước sau.
- **DQL**: dùng mạng neural để xấp xỉ Q-value.
- **REINFORCE**: học trực tiếp policy bằng return.
- **PPO**: policy gradient ổn định hơn nhờ cơ chế clip.
- **AWR**: học policy bằng hồi quy có trọng số theo advantage.

Có thể nhóm ngắn gọn như sau:

- **Nhóm value-based**: SARSA, Q-learning, DQL.
- **Nhóm policy-based**: REINFORCE.
- **Nhóm policy optimization hiện đại**: PPO, AWR.

Khi học các thuật toán này, điều quan trọng nhất không phải là nhớ ngay toàn bộ công thức, mà là hiểu mỗi thuật toán đang cập nhật cái gì, dùng tín hiệu nào để học, và khác nhau ở điểm nào. Khi nắm được điều đó, người học sẽ dễ dàng đọc code, triển khai mô hình và mở rộng sang các phương pháp nâng cao hơn.

VI. Thực hành bài toán Q-learning với Taxi-v3

Bài tập này triển khai **Q-learning** trên môi trường **Taxi-v3**. Mục tiêu là huấn luyện agent học cách điều khiển taxi đón đúng hành khách và đưa đến đúng vị trí đích. Nội dung tập trung vào các phần cốt lõi của bài toán: khởi tạo môi trường, xây dựng Q-table, huấn luyện bằng chiến lược *epsilon-greedy*, và kiểm tra kết quả suy luận sau khi học xong.

Trong bài này, mỗi giá trị

$$Q(s, a)$$

biểu diễn mức độ tốt của hành động a tại trạng thái s . Qua quá trình huấn luyện, bảng Q sẽ được cập nhật dần để phản ánh tốt hơn hành động nào nên chọn ở từng trạng thái.

Bước 1: Khởi tạo môi trường và Q-table

```
1 import gymnasium as gym
2 import numpy as np
3
4 env = gym.make("Taxi-v3", render_mode="rgb_array")
5 state_space = env.observation_space.n
6 action_space = env.action_space.n
7
8 def init_q_table(state_space, action_space):
9     q_table = np.zeros((state_space, action_space))
10    return q_table
```

Môi trường Taxi-v3 có không gian trạng thái và hành động rời rạc, nên có thể lưu trực tiếp giá trị dưới dạng một bảng **Q-table**. Kích thước của bảng là:

$$(\text{state_space}, \text{action_space})$$

Ban đầu, toàn bộ giá trị được khởi tạo bằng 0.

Bước 2: Xây dựng policy chọn hành động

```
1 def greedy_policy(q_table, state):
2     action = np.argmax(q_table[state, :])
3     return action
4
5 def epsilon_greedy_policy(q_table, state, epsilon):
6     rand_n = float(np.random.uniform(0, 1))
7
8     if rand_n > epsilon:
9         action = greedy_policy(q_table, state)
```

```
10     else:
11         action = np.random.choice(q_table.shape[1])
12
13     return action
```

Trong giai đoạn huấn luyện, agent sử dụng chiến lược **epsilon-greedy**. Với xác suất ϵ , một hành động ngẫu nhiên sẽ được chọn để khám phá. Với xác suất còn lại, agent chọn hành động có Q-value lớn nhất để khai thác kiến thức hiện có.

Bước 3: Huấn luyện bằng Q-learning

```
1 def train(
2     env,
3     max_steps,
4     q_table,
5     n_training_episodes,
6     min_epsilon,
7     max_epsilon,
8     decay_rate,
9     lr,
10    gamma
11 ):
12     episode_rewards = []
13
14     for episode in range(n_training_episodes):
15         epsilon = min_epsilon + \
16             (max_epsilon - min_epsilon) * \
17             np.exp(-decay_rate * episode)
18
19         state, info = env.reset()
20         terminated = False
21         truncated = False
22         total_reward = 0.0
23
24         for step in range(max_steps):
25             action = epsilon_greedy_policy(q_table, state, epsilon)
26             new_state, reward, terminated, truncated, info = env.step(action)
27
28             q_table[state, action] = q_table[state, action] + lr * (
29                 reward + gamma * np.max(q_table[new_state]) - q_table[state, action]
30             )
31
32             total_reward += reward
33
34             if terminated or truncated:
35                 break
36
37             state = new_state
38
```

```

39     episode_rewards.append(total_reward)
40
41     return q_table, episode_rewards

```

Công thức cập nhật quan trọng nhất của Q-learning là:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Trong đó:

- α : learning rate.
- γ : discount factor.
- r_{t+1} : reward nhận được sau khi thực hiện hành động.
- $\max_{a'} Q(s_{t+1}, a')$: giá trị tốt nhất có thể đạt được ở trạng thái kế tiếp.

Ý tưởng của công thức trên là cập nhật giá trị hiện tại của hành động sao cho nó tiến gần hơn đến **target** mới, gồm phần thưởng trước mắt và lợi ích tốt nhất trong tương lai.

Ví dụ: giả sử tại một bước huấn luyện có:

$$Q(s_t, a_t) = 5, \quad r_{t+1} = 2, \quad \gamma = 0.95, \quad \max_{a'} Q(s_{t+1}, a') = 6, \quad \alpha = 0.7$$

Khi đó:

$$\text{Target} = 2 + 0.95 \cdot 6 = 7.7$$

Giá trị mới sẽ là:

$$Q(s_t, a_t) \leftarrow 5 + 0.7(7.7 - 5) = 6.89$$

Sau nhiều episode, Q-table dần chứa thông tin tốt hơn về hành động nên chọn ở từng trạng thái.

Bước 4: Hoàn thiện phần còn thiếu

```

1 n_training_episodes = 30000
2 max_steps = 99
3 lr = 0.7
4 gamma = 0.95
5
6 max_epsilon = 1.0
7 min_epsilon = 0.05
8 decay_rate = 0.0005
9
10 q_table = ***Your Code Here***
11 trained_q_table, episode_rewards = ***Your Code Here***

```

Hai dòng trên cần được hoàn thiện như sau:

- Dòng thứ nhất khởi tạo **Q-table** từ số lượng trạng thái và hành động của môi trường.
- Dòng thứ hai gọi hàm **train(...)** với các tham số đã khai báo để thu được Q-table sau huấn luyện và danh sách reward theo từng episode.

Phần này chỉ yêu cầu ghép đúng các thành phần đã xây dựng ở trên, không cần viết thêm thuật toán mới.

Bước 5: Kiểm tra suy luận

```

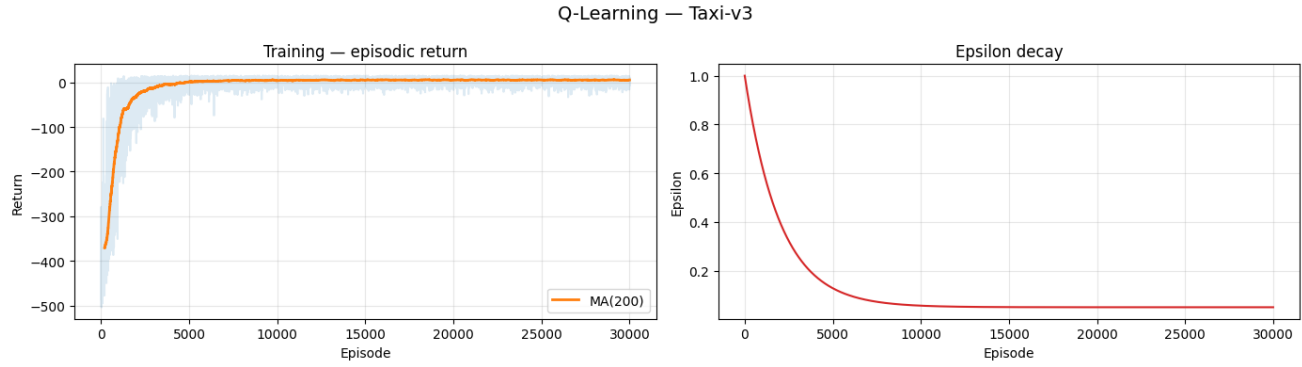
1 def record_video(env, q_table, gif_path, fps=2):
2     images = []
3     terminated = False
4     truncated = False
5
6     state, info = env.reset(seed=np.random.randint(0, 500))
7     img = env.render()
8     images.append(img)
9     total_rewards = 0
10
11     while not (terminated or truncated):
12         action = greedy_policy(q_table, state)
13         state, reward, terminated, truncated, info = env.step(action)
14         total_rewards += reward
15         img = env.render()
16         images.append(img)
17
18     print(f"Total rewards: {total_rewards}")
19     return gif_path

```

Sau khi huấn luyện xong, quá trình suy luận sử dụng **greedy policy**:

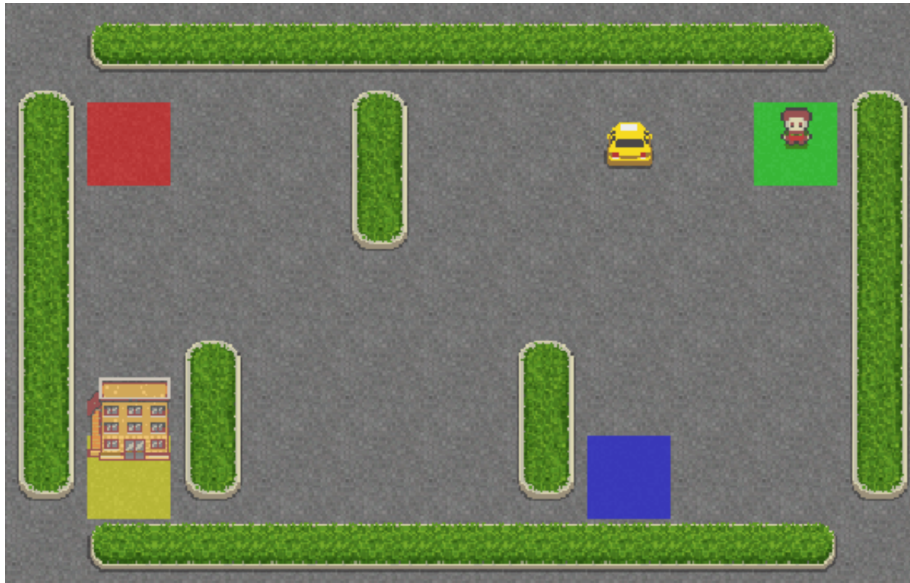
$$a_t = \arg \max_a Q(s_t, a)$$

Lúc này, agent không còn khám phá ngẫu nhiên mà luôn chọn hành động có giá trị lớn nhất trong Q-table. Kết quả suy luận cho thấy agent đã học được cách điều khiển taxi hiệu quả đến mức nào.



Hình 7: Đường cong episodic return và quá trình epsilon decay của Q-learning trên Taxi-v3.

Kết quả huấn luyện cho thấy return tăng nhanh trong giai đoạn đầu và dần ổn định về sau, cho thấy agent đã học được policy tốt hơn theo thời gian. Đường cong epsilon decay cũng cho thấy mức khám phá giảm dần, giúp agent chuyển từ thử nghiệm ngẫu nhiên sang ưu tiên khai thác các hành động có giá trị cao.



Hình 8: Minh họa một trạng thái suy luận của agent trong môi trường Taxi-v3 sau khi huấn luyện.

Ở bước suy luận, agent sử dụng greedy policy để chọn hành động tốt nhất theo Q-table đã học được. Kết quả này giúp quan sát trực quan rằng taxi đã biết di chuyển hợp lý hơn trong môi trường thay vì hành động ngẫu nhiên như ở đầu quá trình huấn luyện.

Tóm lại, bài tập này cần nắm được ba ý chính:

- Q-table lưu giá trị của từng cặp trạng thái – hành động.
- Q-learning cập nhật giá trị đó sau mỗi lần tương tác với môi trường.

- Epsilon-greedy giúp cân bằng giữa khám phá và khai thác trong quá trình huấn luyện.

VII. Thực hành approximate Q-learning với MsPacman

Bài tập này triển khai **approximate Q-learning** trên môi trường **MsPacman**. Khác với bài Taxi-v3 dùng bảng Q cho trạng thái rời rạc, ở đây trạng thái được biểu diễn bằng vector **RAM 128 chiều**, nên không thể lưu trực tiếp một Q-table cho mọi trạng thái. Thay vào đó, bài toán sử dụng một hàm xấp xỉ tuyến tính để ước lượng giá trị:

$$Q(s, a) \approx w_a^\top \phi(s)$$

trong đó $\phi(s)$ là vector đặc trưng của trạng thái và w_a là vector trọng số ứng với hành động a .

Mục tiêu của bài tập là xây dựng đặc trưng đầu vào, huấn luyện trọng số bằng Q-learning với *epsilon-greedy*, rồi kiểm tra kết quả suy luận sau khi học xong.

Bước 1: Khởi tạo môi trường và biểu diễn đặc trưng

```

1 import gymnasium as gym
2 import numpy as np
3
4 ENV_ID = "ALE/MsPacman-v5"
5
6 train_env = gym.make(
7     ENV_ID,
8     obs_type="ram",
9     frameskip=1,
10    repeat_action_probability=0.0,
11 )
12
13 obs_dim = train_env.observation_space.shape[0]
14 n_actions = train_env.action_space.n
15
16 def featurize(obs_ram):
17     x = obs_ram.astype(np.float32) / 255.0
18     return np.concatenate([x, np.array([1.0], dtype=np.float32)])

```

Trong bài này, đầu vào không phải ảnh mà là **RAM state** của trò chơi. Hàm `featurize(...)` chuẩn hoá vector RAM về khoảng $[0, 1]$, sau đó nối thêm một phần tử **bias**. Vì vậy, nếu RAM có 128 chiều thì vector đặc trưng cuối cùng sẽ có:

$$\text{feat_dim} = 128 + 1 = 129$$

Cách biểu diễn này cho phép xây dựng một hàm Q tuyến tính đơn giản nhưng vẫn đủ để minh hoạ ý tưởng **xấp xỉ hàm giá trị**.

Bước 2: Xây dựng hàm Q và policy chọn hành động

```

1 def q_values(weights, phi):
2     return weights @ phi
3
4 def epsilon_greedy(weights, phi, epsilon, action_space):
5     if np.random.rand() < epsilon:
6         return action_space.sample()
7     return int(np.argmax(q_values(weights, phi)))
8
9 feat_dim = obs_dim + 1
10 weights = np.zeros((n_actions, feat_dim), dtype=np.float32)

```

Ở đây, mỗi hành động có một vector trọng số riêng. Toàn bộ tham số được lưu trong ma trận:

$$W \in \mathbb{R}^{n_{\text{actions}} \times \text{feat_dim}}$$

Mỗi hàng của ma trận tương ứng với một hành động. Khi đã có vector đặc trưng $\phi(s)$, giá trị của tất cả các hành động được tính đồng thời bằng:

$$Q(s, \cdot) = W\phi(s)$$

Trong giai đoạn huấn luyện, policy vẫn dùng **epsilon-greedy**: đôi khi chọn ngẫu nhiên để khám phá, còn lại chọn hành động có Q-value lớn nhất.

Bước 3: Huấn luyện approximate Q-learning

```

1 history = []
2 epsilon = EPS_START
3
4 for ep in range(1, NUM_TRAINING + 1):
5     obs, _ = train_env.reset(seed=SEED + ep)
6     phi = featurize(obs)
7     ep_reward = 0.0
8
9     for _ in range(MAX_STEPS):
10        action = ***Your Code Here***
11        next_obs, reward, terminated, truncated, _ = ***Your Code Here***
12        next_phi = ***Your Code Here***
13
14        q_sa = weights[action].dot(phi)
15        target = reward
16        if not (terminated or truncated):
17            target += GAMMA * np.max(q_values(weights, next_phi))
18
19        td_error = target - q_sa
20        weights[action] += ALPHA * td_error * phi
21

```



```

22     ep_reward += reward
23     phi = next_phi
24     if terminated or truncated:
25         break
26
27     history.append({"episode": ep, "reward": ep_reward, "epsilon": epsilon})
28     epsilon = max(EPS_END, epsilon * EPS_DECAY)

```

Phần cốt lõi của bài nằm ở vòng lặp huấn luyện trên. Công thức Q-learning vẫn giữ nguyên ý tưởng như bài Taxi-v3, nhưng bây giờ không cập nhật một ô trong Q-table mà cập nhật trực tiếp **vector trọng số** của hành động đang chọn.

Giá trị hiện tại của cặp trạng thái – hành động là:

$$Q(s_t, a_t) = w_{a_t}^\top \phi(s_t)$$

Target của Q-learning là:

$$y_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$$

Sai số TD là:

$$\delta_t = y_t - Q(s_t, a_t)$$

Và quy tắc cập nhật trọng số là:

$$w_{a_t} \leftarrow w_{a_t} + \alpha \delta_t \phi(s_t)$$

Ý nghĩa của công thức trên là: nếu dự đoán hiện tại còn thấp hơn target, trọng số sẽ được điều chỉnh theo hướng làm tăng giá trị $Q(s_t, a_t)$. Ngược lại, nếu dự đoán quá cao, trọng số sẽ được kéo giảm xuống.

Ví dụ: giả sử tại một bước có:

$$Q(s_t, a_t) = 1.2, \quad r_{t+1} = 0.5, \quad \gamma = 0.99, \quad \max_{a'} Q(s_{t+1}, a') = 2.0$$

Khi đó:

$$y_t = 0.5 + 0.99 \cdot 2.0 = 2.48$$

và:

$$\delta_t = 2.48 - 1.2 = 1.28$$

Nếu α dương, trọng số của hành động hiện tại sẽ được cập nhật theo hướng của $\phi(s_t)$ để tăng giá trị dự đoán lên gần target hơn.

Bước 4: Hoàn thiện phần còn thiếu

```

1 for _ in range(MAX_STEPS):
2     action = ***Your Code Here***
3     next_obs, reward, terminated, truncated, _ = ***Your Code Here***
4     next_phi = ***Your Code Here***

```

Ba dòng trên cần được hoàn thiện như sau:

- Dòng thứ nhất chọn **action** bằng chiến lược **epsilon-greedy** từ trọng số hiện tại, đặc trưng hiện tại và giá trị ϵ .
- Dòng thứ hai thực hiện hành động vừa chọn trên môi trường để lấy **trạng thái kế tiếp**, **reward** và tín hiệu kết thúc episode.
- Dòng thứ ba biến đổi trạng thái kế tiếp thành **vector đặc trưng** bằng hàm `featurize(...)`.

Phần này chỉ yêu cầu nối đúng các thành phần đã được xây dựng ở trên: policy chọn hành động, bước chuyển môi trường, và hàm tạo đặc trưng.

Bước 5: Kiểm tra suy luận

```

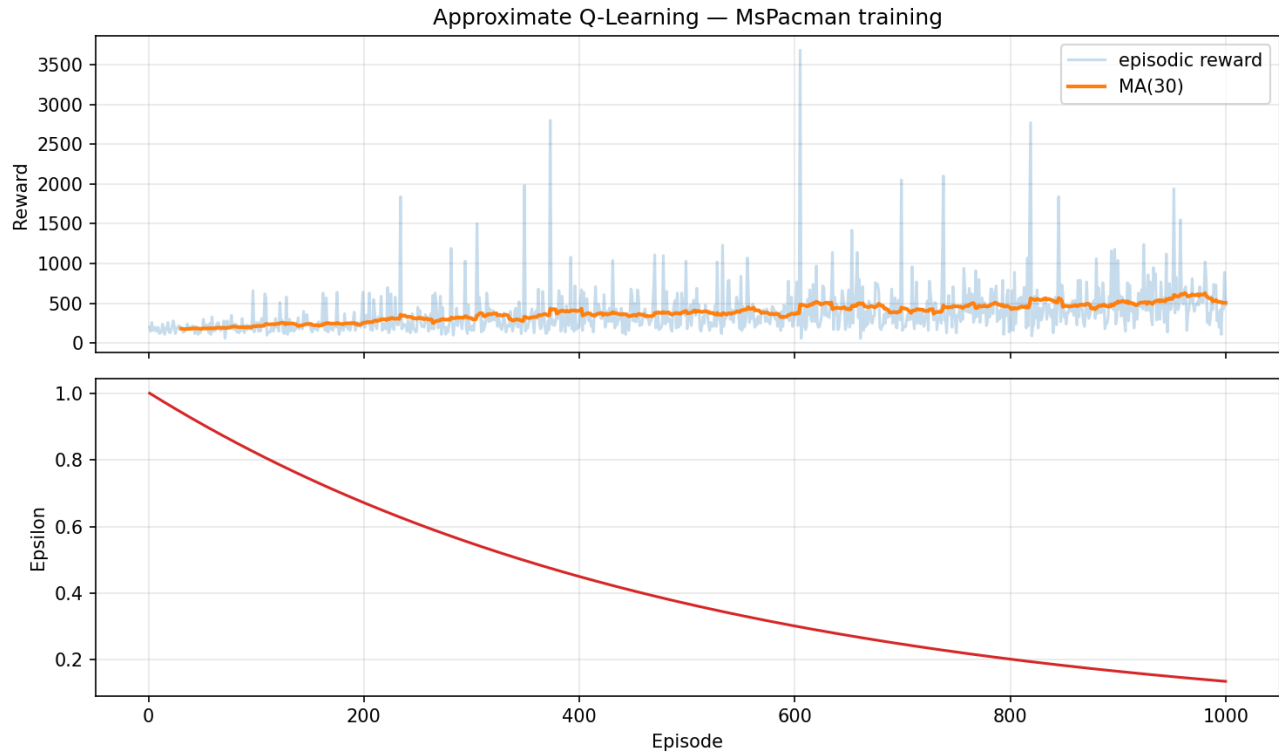
1 obs, _ = eval_env.reset(seed=SEED + 999)
2 phi = featurize(obs)
3 frames = []
4 total_reward = 0.0
5
6 for _ in range(MAX_STEPS):
7     frames.append(eval_env.render())
8     action = int(np.argmax(q_values(weights, phi)))
9     obs, reward, terminated, truncated, _ = eval_env.step(action)
10    total_reward += reward
11    phi = featurize(obs)
12    if terminated or truncated:
13        break

```

Sau khi huấn luyện, quá trình suy luận không còn dùng khám phá ngẫu nhiên mà luôn chọn hành động tốt nhất theo trọng số hiện tại:

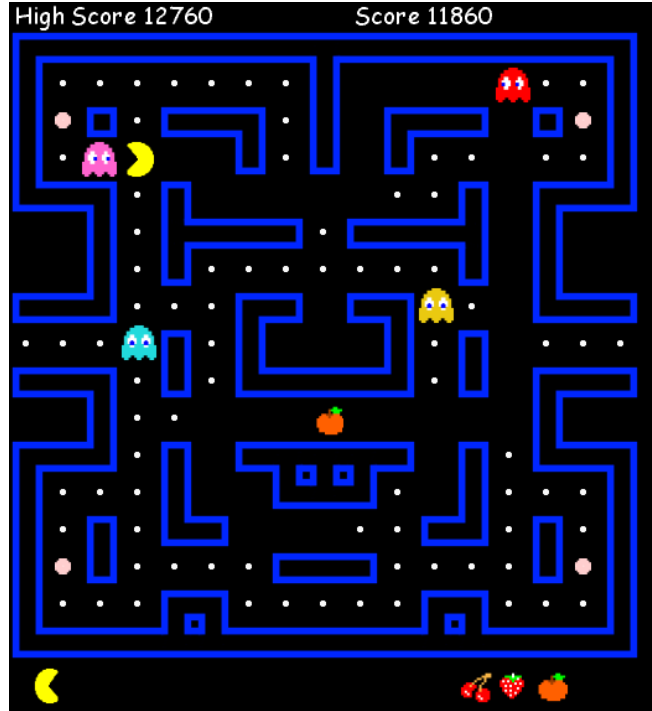
$$a_t = \arg \max_a Q(s_t, a)$$

Kết quả suy luận cho thấy mức độ mà agent đã học được cách điều khiển Pacman dựa trên biểu diễn RAM và hàm Q tuyến tính.



Hình 9: Đường cong reward theo episode và quá trình epsilon decay của approximate Q-learning trên MsPacman.

Kết quả huấn luyện cho thấy reward có xu hướng cải thiện dần theo thời gian, dù vẫn dao động khá mạnh vì môi trường MsPacman có độ phức tạp cao và phần thưởng thay đổi lớn giữa các episode. Đường trung bình trượt giúp thấy rõ hơn xu hướng học đi lên của agent, còn đường epsilon decay cho thấy mức khám phá giảm dần trong suốt quá trình huấn luyện.



Hình 10: Minh hoạ một trạng thái suy luận của agent trong môi trường MsPacman sau khi huấn luyện.

Ở bước suy luận, agent không còn chọn hành động ngẫu nhiên mà sử dụng hàm Q xấp xỉ đã học để chọn hành động tốt nhất tại mỗi trạng thái. Hình minh hoạ cho thấy agent đã có khả năng điều khiển Pacman theo một chiến lược hợp lý hơn nhằm tránh nguy hiểm và thu thập thêm điểm.

Tóm lại, bài tập này cần nắm được ba ý chính:

- Khi không thể dùng Q -table, có thể xấp xỉ $Q(s, a)$ bằng một hàm tham số.
- Trong bài này, hàm xấp xỉ được chọn là mô hình tuyến tính theo vector đặc trưng.
- Quy tắc học vẫn dựa trên Q -learning, nhưng phần được cập nhật là trọng số thay vì một ô trong bảng.

VIII. Thực hành REINFORCE với LunarLander-v3

Bài tập này triển khai **REINFORCE** trên môi trường **LunarLander-v3**. Khác với các bài Q -learning trước đó, ở đây không học trực tiếp giá trị của từng hành động, mà học một **policy** cho biết xác suất chọn hành động trong mỗi trạng thái. Mục tiêu là huấn luyện agent hạ cánh ổn định bằng cách tối ưu trực tiếp tham số của policy network.

Trong bài này, policy được mô hình hoá bằng một mạng neural nhận trạng thái đầu vào và trả ra phân phối xác suất trên các hành động:

$$\pi_{\theta}(a \mid s)$$

Trong đó, θ là các tham số của mạng. Từ phân phối này, một hành động sẽ được lấy mẫu để tương tác với môi trường.

Bước 1: Xây dựng policy network

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 from torch.distributions import Categorical
5
6 class Policy(nn.Module):
7     def __init__(self, s_size, a_size, h_size):
8         super(Policy, self).__init__()
9         self.fc1 = nn.Linear(s_size, h_size)
10        self.fc2 = nn.Linear(h_size, h_size * 2)
11        self.fc3 = nn.Linear(h_size * 2, a_size)
12
13    def forward(self, x):
14        x = F.relu(self.fc1(x))
15        x = F.relu(self.fc2(x))
16        x = self.fc3(x)
17
18        return F.softmax(x, dim=1)
19
20    def act(self, state):
21        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
22        probs = self.forward(state).cpu()
23        m = Categorical(probs)
24        action = m.sample()
25
26        return action.item(), m.log_prob(action)

```

Mạng Policy nhận một trạng thái và sinh ra vector xác suất trên các hành động. Hàm `act(...)` không chỉ trả về hành động được lấy mẫu mà còn trả về **log-probability** của hành động đó. Giá trị này rất quan trọng vì sẽ được dùng trực tiếp trong hàm mất mát của REINFORCE.

Ý tưởng chính là:

$$a_t \sim \pi_{\theta}(\cdot \mid s_t)$$

Tức là hành động không được chọn theo kiểu **argmax** cố định, mà được lấy mẫu từ phân phối xác suất do policy sinh ra.

Bước 2: Huấn luyện bằng REINFORCE

```
1 def reinforce(
2     policy,
3     optimizer,
4     n_training_episodes,
5     max_steps,
6     gamma,
7     print_every
8 ):
9     scores_deque = deque(maxlen=100)
10    scores = []
11
12    for i_episode in range(1, n_training_episodes + 1):
13        saved_log_probs = []
14        rewards = []
15        state, _ = env.reset()
16
17        for t in range(max_steps):
18            action, log_prob = policy.act(state)
19            saved_log_probs.append(log_prob)
20            state, reward, terminated, truncated, info = env.step(action)
21            rewards.append(reward)
22            if terminated or truncated:
23                break
24
25        scores_deque.append(sum(rewards))
26        scores.append(sum(rewards))
27
28        returns = deque(maxlen=max_steps)
29        n_steps = len(rewards)
30
31        for t in range(n_steps)[::-1]:
32            disc_return_t = returns[0] if len(returns) > 0 else 0
33            returns.appendleft(gamma * disc_return_t + rewards[t])
34
35        eps = np.finfo(np.float32).eps.item()
36
37        returns = torch.tensor(returns)
38        returns = (returns - returns.mean()) / (returns.std() + eps)
39
40        policy_loss = []
41        for log_prob, disc_return in zip(saved_log_probs, returns):
42            policy_loss.append(-log_prob * disc_return)
43        policy_loss = torch.cat(policy_loss).sum()
44
45        optimizer.zero_grad()
46        policy_loss.backward()
47        optimizer.step()
48
49    return scores
```

REINFORCE học theo ý tưởng rất trực tiếp: nếu một hành động nằm trên quỹ đạo có return cao, thì xác suất chọn hành động đó trong tương lai nên được tăng lên. Return tại thời điểm t được tính bởi:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

Gradient cốt lõi của REINFORCE có dạng:

$$\nabla J(\theta) = \mathbb{E} \left[G_t \nabla \log \pi_{\theta}(a_t | s_t) \right]$$

Trong code, điều này được chuyển thành hàm mất mát:

$$\mathcal{L} = - \sum_t \log \pi_{\theta}(a_t | s_t) G_t$$

Vì mục tiêu là **tối đa hoá** return, nên khi viết dưới dạng loss để tối ưu bằng gradient descent, cần thêm dấu âm ở phía trước. Trong bài này, các return còn được **chuẩn hoá** trước khi cập nhật để giúp quá trình học ổn định hơn.

Ví dụ: giả sử tại một bước có:

$$\log \pi_{\theta}(a_t | s_t) = -0.4, \quad G_t = 3.0$$

Khi đó đóng góp vào loss là:

$$-(-0.4) \cdot 3.0 = 1.2$$

Nếu một hành động có return lớn, gradient sẽ điều chỉnh tham số theo hướng làm tăng xác suất của hành động đó. Ngược lại, nếu return thấp, tác động cập nhật sẽ yếu hơn hoặc theo hướng giảm xác suất.

Bước 3: Hoàn thiện phần còn thiếu

```

1 device = torch.device(
2     "cuda" if torch.cuda.is_available() else "cpu"
3 )
4
5 policy = ***Your Code Here***
6
7 optimizer = ***Your Code Here***

```

Hai dòng trên cần được hoàn thiện theo đúng vai trò của chúng:

- Dòng thứ nhất cần **khởi tạo policy network** từ kích thước trạng thái, số lượng hành động và kích thước hidden layer, sau đó đưa mô hình lên **device**.
- Dòng thứ hai cần **khởi tạo optimizer** để cập nhật tham số của policy với learning rate đã khai báo.

Hướng đi đúng là tạo một đối tượng từ lớp `Policy(...)` và dùng một optimizer phổ biến như Adam trên `policy.parameters()`. Phần này chỉ yêu cầu nối đúng các thành phần đã có sẵn ở phía trên.

Bước 4: Gọi huấn luyện

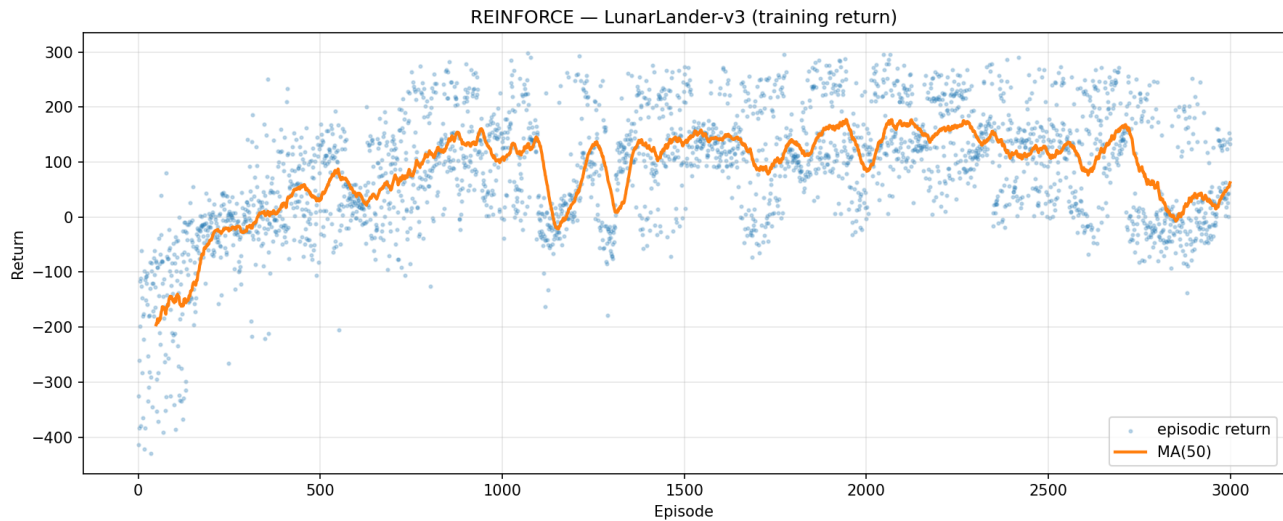
```
1 scores = reinforce(  
2     policy,  
3     optimizer,  
4     n_training_episodes,  
5     max_steps,  
6     gamma,  
7     print_every=100  
8 )
```

Sau khi có policy và optimizer, hàm `reinforce(...)` sẽ chạy nhiều episode để thu thập trajectory, tính return, xây dựng `policy_loss`, rồi cập nhật tham số mạng. Kết quả trả về là danh sách `scores` để theo dõi training return theo thời gian.

Bước 5: Đánh giá và suy luận

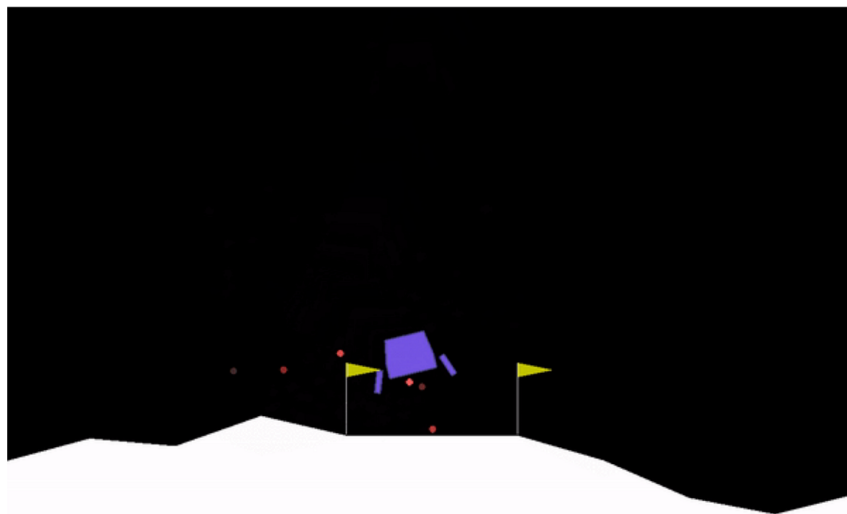
```
1 def evaluate_agent(  
2     env,  
3     max_steps,  
4     n_eval_episodes,  
5     policy,  
6     seed  
7 ):  
8     episode_rewards = []  
9     for episode in range(n_eval_episodes):  
10         if seed:  
11             state, _ = env.reset(seed=seed[episode])  
12         else:  
13             state, _ = env.reset()  
14             total_rewards_ep = 0  
15  
16         for step in range(max_steps):  
17             action, _ = policy.act(state)  
18             new_state, reward, terminated, truncated, info = env.step(action)  
19             total_rewards_ep += reward  
20  
21             if terminated or truncated:  
22                 break  
23  
24             state = new_state  
25             episode_rewards.append(total_rewards_ep)  
26  
27     mean_reward = np.mean(episode_rewards)  
28     std_reward = np.std(episode_rewards)  
29  
30     return mean_reward, std_reward
```


Ở bước đánh giá, agent chạy nhiều episode để đo **mean reward** và **standard deviation**. Đây là cách kiểm tra policy đã học được ổn định đến mức nào. Sau đó, phần `record_video(...)` được dùng để lưu lại quá trình suy luận thành GIF nhằm quan sát trực quan hành vi của agent trong môi trường.



Hình 11: Đường cong training return của REINFORCE trên LunarLander-v3.

Kết quả huấn luyện cho thấy return có xu hướng cải thiện rõ rệt so với giai đoạn đầu, từ mức âm khá lớn lên vùng dương ổn định hơn. Tuy nhiên, do REINFORCE là một phương pháp policy gradient cơ bản với phương sai cao, đường return vẫn còn dao động mạnh giữa các episode. Đường trung bình trượt giúp quan sát rõ hơn xu hướng học tổng thể của agent theo thời gian.



Hình 12: Minh họa một trạng thái suy luận của agent trong môi trường LunarLander-v3 sau khi huấn luyện.

Trong giai đoạn suy luận, agent sử dụng policy đã học để chọn hành động dựa trên phân phối xác suất đầu ra của mạng. Hình minh họa cho thấy agent đã học được một phần hành vi điều khiển tàu đổ bộ hợp lý hơn, hướng tới mục tiêu hạ cánh an toàn trong vùng giữa hai cột mốc.

Tóm lại, bài tập này cần nắm được ba ý chính:

- REINFORCE học trực tiếp policy thay vì học Q-table.
- Tín hiệu cập nhật được xây từ **log-probability** của hành động và **return** của quỹ đạo.
- Phần cần điền tập trung vào **khởi tạo policy network** và **optimizer** đúng với các thành phần đã khai báo.

IX. Thực hành PPO với Acrobot-v1

Bài tập này triển khai **Proximal Policy Optimization (PPO)** trên môi trường **Acrobot-v1**. Agent được xây dựng theo kiểu **actor-critic**: actor sinh phân phối hành động, còn critic ước lượng giá trị trạng thái. Trong quá trình huấn luyện, agent thu thập dữ liệu từ nhiều môi trường song song, tính **advantage**, rồi cập nhật policy bằng hàm mục tiêu **clipped objective** để quá trình học ổn định hơn.

Trong bài này, phần quan trọng nhất cần nắm là:

- Actor sinh logits để chọn hành động.
- Critic ước lượng $V(s)$ để hỗ trợ tính advantage.
- PPO không cập nhật policy quá mạnh trong một lần học, mà dùng cơ chế **clip** để giới hạn mức thay đổi.

Bước 1: Xây dựng agent actor-critic

```
1 class Agent(nn.Module):
2     def __init__(self, envs):
3         super().__init__()
4         obs_dim = int(np.prod(envs.single_observation_space.shape))
5         n_act = envs.single_action_space.n
6
7         self.critic = nn.Sequential(
8             layer_init(nn.Linear(obs_dim, 64)),
9             nn.Tanh(),
10            layer_init(nn.Linear(64, 64)),
11            nn.Tanh(),
12            layer_init(nn.Linear(64, 1), std=1.0),
13        )
14
15        self.actor = nn.Sequential(
16            layer_init(nn.Linear(obs_dim, 64)),
17            nn.Tanh(),
```

```

18         layer_init(nn.Linear(64, 64)),
19         nn.Tanh(),
20         layer_init(nn.Linear(64, n_act), std=0.01),
21     )
22
23     def get_value(self, x):
24         return self.critic(x)
25
26     def get_action_and_value(self, x, action=None):
27         logits = self.actor(x)
28         dist = Categorical(logits=logits)
29         if action is None:
30             action = dist.sample()
31         return action, dist.log_prob(action), dist.entropy(), self.critic(x)

```

Agent gồm hai phần:

- **Actor:** nhận trạng thái đầu vào và sinh logits cho các hành động.
- **Critic:** nhận cùng trạng thái đó và ước lượng giá trị $V(s)$.

Nhờ đó, tại mỗi trạng thái s_t , agent vừa có thể chọn hành động, vừa có thể ước lượng mức tốt của trạng thái hiện tại.

Bước 2: Tính advantage và cập nhật PPO

```

1 delta = rewards[t] + args.gamma * nextvalues * nextnonterminal - values[t]
2
3 advantages[t] = lastgaelam = (
4     delta + args.gamma * args.gae_lambda * nextnonterminal * lastgaelam
5 )
6
7 ratio = (newlogp - flat_logp[mb]).exp()
8
9 pg1 = -mb_adv * ratio
10 pg2 = -mb_adv * torch.clamp(ratio, 1 - args.clip_coef, 1 + args.clip_coef)
11 pg_loss = torch.max(pg1, pg2).mean()
12
13 loss = pg_loss - args.ent_coef * ent + args.vf_coef * v_loss

```

Phần trên là lõi của PPO. Trước hết, bài dùng **GAE** để tính advantage. Sai số TD tại bước t là:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Advantage được cộng dồn theo:

$$\hat{A}_t = \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + \dots$$

Sau đó, PPO dùng tỉ số giữa policy mới và policy cũ:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

Hàm mục tiêu chính của PPO là:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

Ý nghĩa của phần `clip` là không cho policy mới lệch quá xa policy cũ chỉ sau một lần cập nhật. Điều này giúp huấn luyện ổn định hơn so với policy gradient cơ bản.

Ngoài policy loss, bài còn có:

- **Value loss**: giúp critic học tốt hơn.
- **Entropy**: khuyến khích policy còn đủ mức khám phá.

Tổng loss được viết lại dưới dạng tối ưu hoá là:

$$\mathcal{L} = \mathcal{L}_{\text{policy}} - c_1 \mathcal{H} + c_2 \mathcal{L}_{\text{value}}$$

Bước 3: Hoàn thiện phần còn thiếu

```
1 args = ***Your Code Here***
2 ***Your Code Here***
```

Phần còn thiếu nằm ở khối chạy train cuối notebook. Ảnh gợi ý cho thấy có hai dòng cần hoàn thiện:

- Dòng thứ nhất tạo đối tượng **args** bằng cách gọi **build_config(...)** với một số tham số cấu hình train như tổng số bước học, tên thí nghiệm, thư mục lưu kết quả, seed, và số episode đánh giá.
- Dòng thứ hai gọi hàm **train_ppo(args)** để bắt đầu huấn luyện.

Phần này chỉ yêu cầu ghép đúng hai thành phần đã có sẵn trong notebook:

- Hàm **build_config(...)** để tạo cấu hình.
- Hàm **train_ppo(...)** để chạy toàn bộ pipeline PPO.

Khi điền phần này, cần chú ý:

- **total_timesteps** điều khiển độ dài huấn luyện.
- **exp_name** và **results_dir** giúp phân biệt thí nghiệm.
- **seed** và **eval_episodes** dùng để kiểm soát đánh giá.

Bước 4: Đọc kết quả sau huấn luyện

```

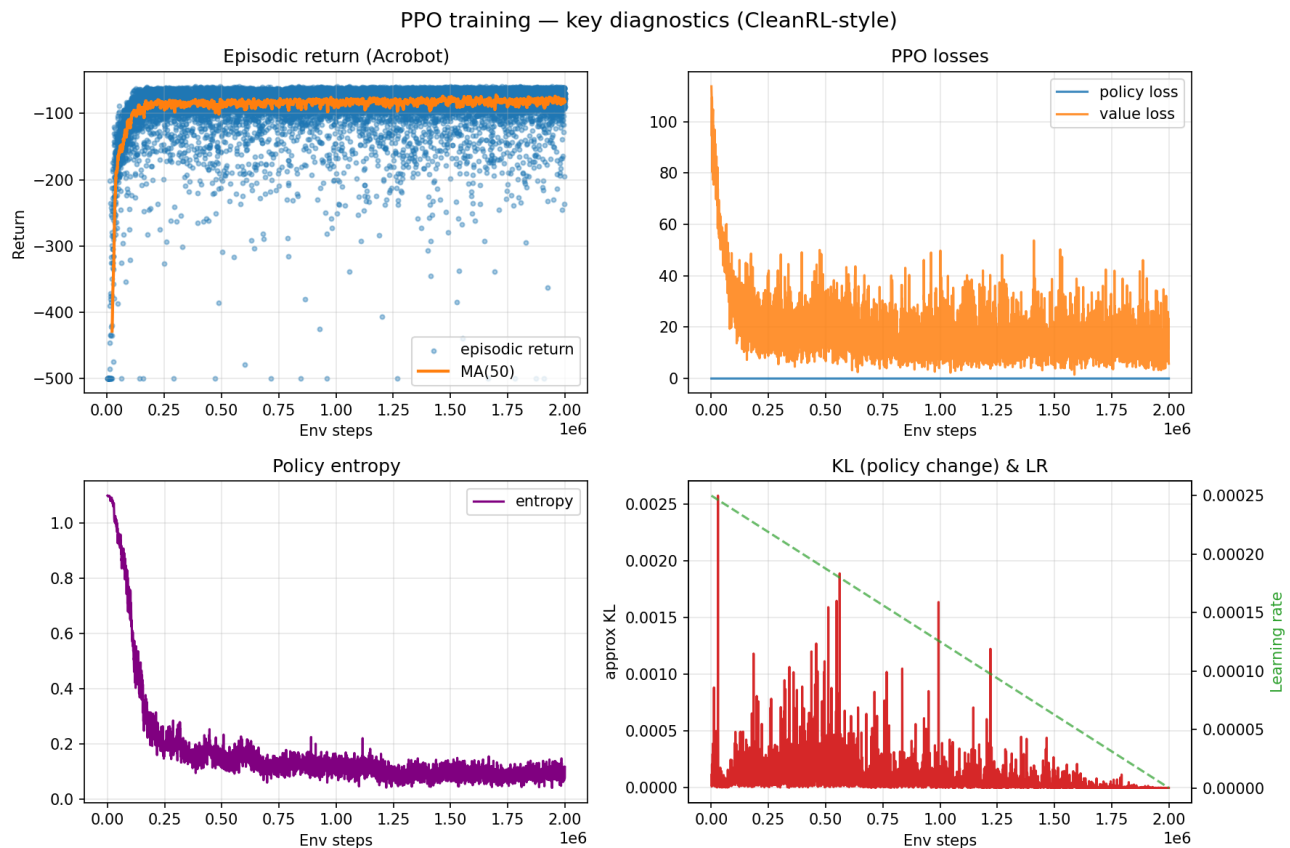
1 results = Path(args.results_dir).resolve()
2 summary_path = results / "training_summary.json"
3
4 display(Image(filename=str(results / "training_curves.png")))
5 display(Image(filename=str(results / "inference_demo.gif")))

```

Sau khi train xong, notebook tự lưu các kết quả quan trọng:

- **training_curves.png**: gồm return, policy loss, value loss, entropy và KL.
- **inference_demo.gif**: mô phỏng quá trình suy luận của agent.
- **training_summary.json**: tóm tắt kết quả đánh giá cuối cùng.

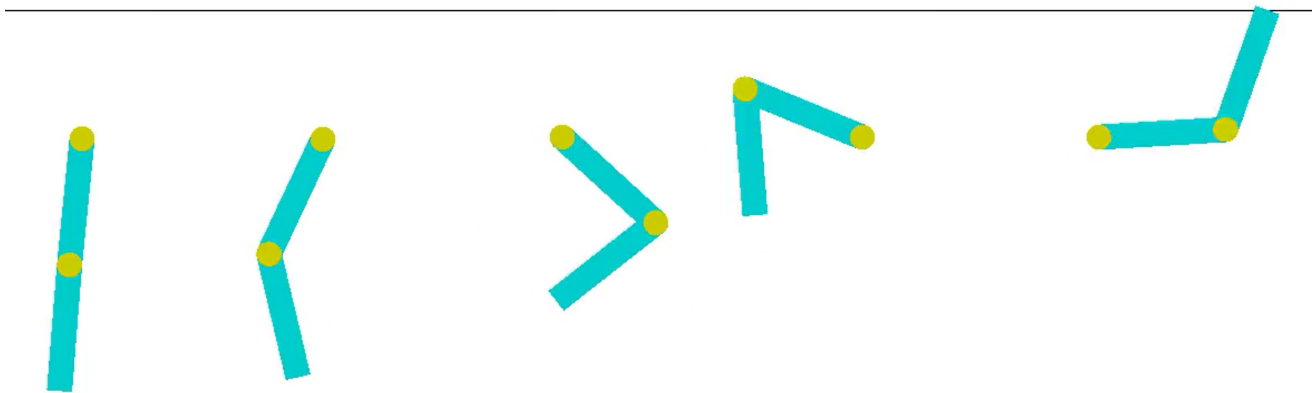
Nhờ đó, có thể kiểm tra không chỉ return, mà còn theo dõi mức thay đổi của policy trong suốt quá trình huấn luyện.



Hình 13: Các biểu đồ chẩn đoán chính trong quá trình huấn luyện PPO trên Acrobot-v1.

Kết quả huấn luyện cho thấy episodic return được cải thiện nhanh trong giai đoạn đầu rồi dần ổn định hơn về sau, phản ánh việc policy học được cách điều khiển hệ Acrobot hiệu quả hơn. Đồng thời, value loss giảm mạnh ở đầu quá trình huấn luyện rồi dao động trong một khoảng hẹp hơn, cho thấy critic dần học được ước lượng giá trị tốt hơn.

Biểu đồ entropy giảm dần theo thời gian, cho thấy policy chuyển từ giai đoạn khám phá nhiều sang hành vi chắc chắn hơn khi đã học được chiến lược tốt. Biểu đồ approximate KL duy trì ở mức nhỏ, trong khi learning rate giảm dần theo lịch annealing, phản ánh đúng tinh thần của PPO là cập nhật policy từng bước nhỏ để giữ quá trình huấn luyện ổn định.



Hình 14: Minh họa một số trạng thái suy luận của agent trong môi trường Acrobot-v1 sau khi huấn luyện.

Hình trên minh họa các trạng thái của hệ Acrobot trong quá trình suy luận. Sau khi huấn luyện, agent đã học được cách tạo dao động và tích lũy động năng để đưa đầu thanh vượt lên vị trí mục tiêu phía trên. Đây là đặc trưng quan trọng của bài toán Acrobot, vì agent không thể đưa hệ lên đích chỉ bằng một hành động trực tiếp mà phải học chiến lược điều khiển theo chuỗi bước liên tiếp.

Tóm lại, bài tập này cần nắm được ba ý chính:

- PPO dùng actor-critic và advantage để cập nhật policy.
- Cơ chế **clip** là thành phần cốt lõi giúp huấn luyện ổn định.
- Phần cần điền nằm ở khối cấu hình train: tạo **args** đúng cách rồi gọi **train_ppo(args)**.

X. Câu hỏi trắc nghiệm

1. (Tính toán – State-value function)

Xét một trạng thái s với policy π chọn ba hành động như sau:

$$\pi(a_1 | s) = 0.2, \quad \pi(a_2 | s) = 0.5, \quad \pi(a_3 | s) = 0.3$$

Giả sử các action-value tương ứng là:

$$Q^\pi(s, a_1) = 2, \quad Q^\pi(s, a_2) = 6, \quad Q^\pi(s, a_3) = 8$$

Khi đó, giá trị trạng thái $V^\pi(s)$ bằng bao nhiêu?

- A. 5.2
- B. 5.8
- C. 6.1
- D. 6.4

2. (Tính toán – Action-value function)

Xét một policy π . Tại trạng thái s , nếu agent chọn hành động a , environment có thể chuyển như sau:

- Với xác suất 0.7, agent nhận reward 3 và chuyển sang s_1 .
- Với xác suất 0.3, agent nhận reward 1 và chuyển sang s_2 .

Biết rằng:

$$V^\pi(s_1) = 4, \quad V^\pi(s_2) = 2, \quad \gamma = 0.9$$

Khi đó, $Q^\pi(s, a)$ bằng bao nhiêu?

- A. 5.46
- B. 5.02
- C. 4.98
- D. 6.12

3. (Tính toán – Advantage)

Tại một trạng thái s , giả sử:

$$Q^\pi(s, a) = 5.4, \quad V^\pi(s) = 6.1$$

Khi đó advantage của hành động a tại trạng thái s là:

- A. 0.7
- B. 1.5
- C. -0.7
- D. -1.5

4. (Tính toán – Monte Carlo Return)

Trong một episode, tại thời điểm t , agent quan sát được chuỗi reward tương lai:

$$r_{t+1} = 2, \quad r_{t+2} = -1, \quad r_{t+3} = 4$$

Giả sử hệ số chiết khấu là:

$$\gamma = 0.5$$

Khi đó Monte Carlo return G_t bằng bao nhiêu?

- A. 2.5
- B. 3.0
- C. 3.5
- D. 4.5

5. (Tính toán – TD(0) Update)

Giả sử tại một bước học, critic hoặc value estimator hiện có:

$$V(s_t) = 3.5, \quad V(s_{t+1}) = 5.0$$

Agent nhận được:

$$r_{t+1} = 2, \quad \gamma = 0.9, \quad \alpha = 0.4$$

Nếu dùng TD(0), giá trị cập nhật mới của $V(s_t)$ là bao nhiêu?

- A. 4.1
- B. 4.3
- C. 4.5
- D. 4.7

6. (Tính toán – Generalized Advantage Estimation)

Giả sử tại ba bước liên tiếp, ta có các TD error:

$$\delta_t = 1.0, \quad \delta_{t+1} = 0.5, \quad \delta_{t+2} = -0.5$$

Biết rằng:

$$\gamma = 0.9, \quad \lambda = 0.8$$

Dùng công thức GAE rút gọn:

$$\hat{A}_t \approx \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2}$$

Khi đó, $\hat{A}_t$ gần nhất với giá trị nào?

- A. 0.8400
- B. 0.9808
- C. 1.1008
- D. 1.3600

7. (Suy luận flow – Q-learning)

Trong Q-learning, agent đang ở trạng thái s_t , chọn hành động a_t , nhận r_{t+1} , rồi chuyển sang s_{t+1} . Giả sử tại s_{t+1} , do cơ chế ϵ -greedy, agent thực tế sẽ thử một hành động ngẫu nhiên a_{t+1}^{explore} , nhưng bảng Q hiện tại lại cho thấy một hành động khác a^* có giá trị lớn nhất.

Nếu flow cài đặt đúng theo **Q-learning chuẩn**, đại lượng nào phải xuất hiện trong target cập nhật?

- A. $Q(s_{t+1}, a_{t+1}^{\text{explore}})$, vì đó là hành động thực sự được chọn tiếp theo.
- B. Giá trị trung bình của mọi action tại s_{t+1} , để tránh thiên lệch.
- C. $\max_{a'} Q(s_{t+1}, a')$, bất kể hành động tiếp theo thật sự có được chọn hay không.
- D. Chỉ dùng reward r_{t+1} , vì Q-learning không bootstrap.

8. (Suy luận flow – PPO)

Trong PPO, với một sample (s_t, a_t) , ta xét tỉ số:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

Giả sử ở một bước cập nhật, ta có:

$$r_t(\theta) = 1.6, \quad \hat{A}_t > 0, \quad \epsilon = 0.2$$

Nếu flow PPO cài đặt đúng theo clipped objective, điều gì hợp lý nhất sẽ xảy ra?

- A. Vì $\hat{A}_t > 0$, luôn dùng trực tiếp $1.6 \hat{A}_t$ để tăng xác suất hành động nhanh nhất có thể.
- B. PPO sẽ dùng nhánh bị chặn bởi $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) = 1.2$, nhằm tránh cập nhật quá mạnh.
- C. PPO sẽ đổi dấu advantage thành âm để trừng phạt policy mới.

D. Sample này bị loại hoàn toàn khỏi batch vì $r_t(\theta)$ vượt khỏi khoảng clip.

9. (Suy luận flow – PPO cho LLM với verifier reward)

Trong một bài toán RLHF hoặc RLVR cho LLM, giả sử model sinh ra một câu trả lời y cho prompt x . Sau đó, một verifier chấm kết quả và trả về reward thấp. Critic hoặc baseline cũng cho biết completion này có **advantage âm**:

$$\hat{A}(x, y) < 0$$

Nếu flow PPO cài đặt đúng, phát biểu nào sau đây là hợp lý nhất?

- A. Model nên tăng xác suất của completion đó, vì câu trả lời đã thật sự được sinh ra.
- B. Model nên bỏ qua reward từ verifier và chỉ giữ nguyên theo xác suất của policy cũ.
- C. Model không thể cập nhật vì reward đến ở mức toàn chuỗi chứ không phải từng token.
- D. Model nên giảm xu hướng sinh lại completion kiểu đó, vì advantage âm cho thấy completion này kém hơn baseline dưới objective hiện tại.

10. (Tính toán và suy luận flow – GRPO cho LLM)

Trong GRPO cho LLM, với cùng một prompt, model sinh ra nhiều completion trong cùng một nhóm. Thay vì dùng một critic riêng để ước lượng $V(s)$, ta có thể dùng **relative advantage** bên trong nhóm.

Giả sử với cùng một prompt, bốn completion nhận reward:

$$r_1 = 0.9, \quad r_2 = 0.6, \quad r_3 = 0.3, \quad r_4 = 0.2$$

Dùng phiên bản đơn giản:

$$A_i = r_i - \bar{r}$$

trong đó $\bar{r}$ là reward trung bình của nhóm. Với completion thứ ba, phát biểu nào đúng nhất?

- A. $A_3 = +0.2$, nên completion thứ ba được khuyến khích mạnh hơn trung bình.
- B. $A_3 = -0.2$, nên completion thứ ba bị đánh giá kém hơn mức trung bình của nhóm và có xu hướng bị giảm xác suất.
- C. $A_3 = 0$, nên completion thứ ba đúng bằng mức trung bình và không tạo tín hiệu học.
- D. $A_3 = -0.5$, nên prompt này phải bị loại khỏi huấn luyện.

Phụ lục

1. **Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải [tại đây](#).
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải [tại đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,4K thành viên



Hình 15: Hình ảnh group facebook AIO Q&A